Unit 1 — The Whole Thing

Coordinates place things. Variables move them. Offsets move them together. Events let a person change them. The console tells you why it didn't work.

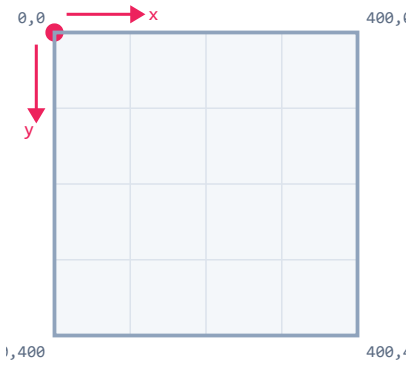
The canvas LESSON 1

Every point on the canvas is two numbers, x first then y.

- x grows going **right**
- y grows going **down**

> **The origin (0, 0) is the top-left corner**, not the middle. A screen fills pixels starting from that corner, so that is where counting starts. A maths graph has the origin in the middle and y going up — this is a different grid.

WHERE	COORDINATE
top-left	`0, 0`
top-right	`width, 0`
bottom-left	`0, height`
bottom-right	`width, height`
the middle	`width/2, height/2`



a 400 × 400 canvas

> **Use `width` and `height` instead of typing numbers.** `line(0, 200, 400, 200)` and `line(0, height/2, width, height/2)` look identical on a 400 × 400 canvas. Change the canvas to 600 × 600 and only the second one is still in the middle. Coordinates can be worked out — they do not have to be typed.

Sketch structure

```
function setup() {
  createCanvas(400, 400);   // runs ONCE, at the start
  rectMode(CENTER);         // settings live here too
}

function draw() {
  background(24, 38, 60);   // runs about 60 times per second
  // every shape goes in here, in order
}
```

`createCanvas` is what makes `width` and `height` have their values. You never set them yourself — you just use them.

Shapes — signatures at a glance LESSONS 1–3

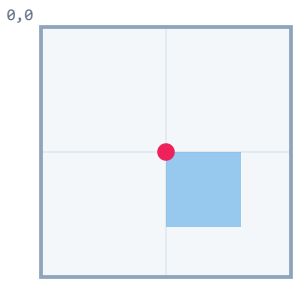
CALL	WHAT THE NUMBERS MEAN
`line(x1, y1, x2, y2)`	two points — where it starts, where it ends
`rect(x, y, w, h)`	one corner + the size. **Last two are a size, not a corner**
`triangle(x1, y1, x2, y2, x3, y3)`	three corners, closes itself
`ellipse(x, y, w, h)`	center + the size. Not a radius

`circle(x, y, d)`	center + diameter. Cannot be squashed — that is what ellipse is for
`background(gray)`	repaints the whole canvas. Goes first, or nothing shows
`background(r, g, b)`	same, in color
`fill(gray)`	one number, 0–255
`fill(r, g, b)`	three numbers, each 0–255
`noFill()`	outline only
`stroke(gray)` · `noStroke()`	the outline color, or none
`strokeWeight(n)`	how thick the outline is

> **To find a rectangle's edges, add.**
> `rect(100, 150, 200, 80)`
> right edge = 100 + 200 = **300** · bottom edge = 150 + 80 = **230**

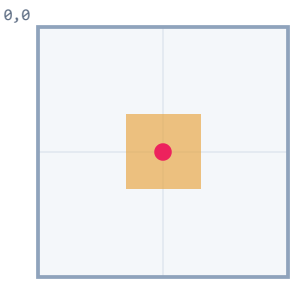
rectMode — what the first two numbers mean LESSON 3

`rectMode(CORNER)` — default



the dot is the corner

`rectMode(CENTER)`



the dot is the middle

Both pictures are `rect(100, 100, 60, 60)`. Only the mode changed. Put `rectMode(CENTER);` in `setup()` once — every `rect()` after it obeys. **Ellipses are already CENTER**, which is why they have always behaved the way you expected.

Draw order LESSON 2

Later shapes cover earlier ones. Reading your `draw()` top to bottom is the order things get painted — which is the same as distance from the viewer.

> **A shape that vanished is usually *covered*, not gone.** Move it lower in `draw()` and it reappears. A shape drawn *above* `background()` really does disappear — erased every frame.

Color LESSONS 1–2

One number is gray, 0 to 255. Three numbers are red, green, blue — each 0 to 255. Light **adds**: red + green is yellow, and all three up is white.

CALL	RESULT
`fill(200)`	gray
`fill(255, 0, 0)`	red
`fill(0, 255, 0)`	green
`fill(0, 0, 255)`	blue
`fill(255, 255, 0)`	yellow
`fill(0, 0, 0)`	black
`fill(255, 255, 255)`	white



one number, black to white

`fill()` is a setting, not part of a shape. It applies to everything drawn after it, until you call it again. One `fill()` and four shapes gives you four shapes the same color. `stroke()` and `rectMode()` work the same way.

Variables LESSON 4A

```
let headY = 190;
//  \__/  \___/   \_/
//  make  the      what goes
//  a name name    in it
```

Use one when the number has to change while the sketch runs, or the same number appears in several places.

`let` goes at the top, outside `draw()`. Inside `draw()` it is created fresh sixty times a second, so it can never grow — your sketch will sit perfectly still, and the cause will be four lines away from where you are looking.

Read `=` as "becomes", not "equals". `y = y + 1` means: take what is in `y`, add one, put it back. In maths that sentence has no solution. Here it is an instruction, not a claim.

The draw loop — why one line moves more than once LESSON 4A

```
function draw() {
  background(24, 38, 60);
  circle(200, y, 60);
  y = y + 1;
}
```

`draw()` is not run once. It runs about **60 times every second**, forever, until you stop the sketch. One `y = y + 1` is sixty pixels of movement per second.

Speed is just how much you add.
`y = y + 1` slow · `y = y + 6` fast
`y = y - 2` goes **up** — y grows downward.

FRAME	Y
1	101
2	102
3	103
…	…
60	160

one second later

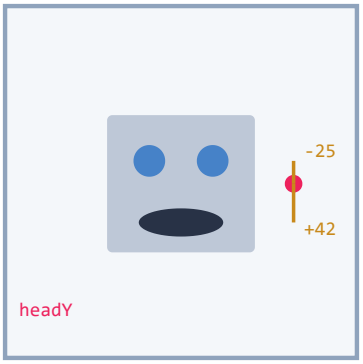
`background()` is at the top of `draw()` for the same reason. Without it, every frame paints on top of the last and a moving shape leaves a trail.

Offsets — moving a whole object LESSONS 4A–4B

Pick **one point**, put it in a variable, and write every other shape as a distance from it — left or right, up or down.

```
rect(200, headY, 160, 150);
circle(165, headY - 25, 34);
circle(235, headY - 25, 34);
ellipse(200, headY + 42, 90, 30);
```

Working out an offset. Subtract the starting value of the variable from where the shape currently sits. The window sat at `125` and `landerY` starts at `40`, so `125 - 40 = 85` — the window is `landerY + 85`.



the dot is the anchor

Only positions move. Sizes never do. `160`, `150`, `34` stay exactly as they are. A drone does not get bigger when it flies right — if yours stretches sideways, you converted a size by mistake.

The corner test. Near the middle of the canvas, a part that is twenty pixels off still looks about right. Move the mouse into all four corners — anything left behind still has a fixed number.

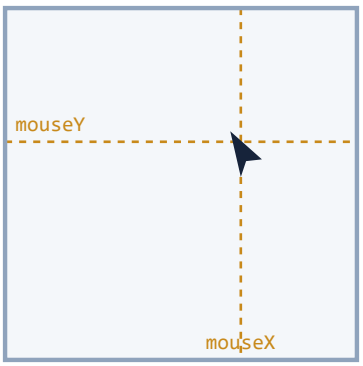
A point you do not control — mouseX and mouseY LESSON 4B

```
circle(mouseX, mouseY, 40);
```

Two variables p5 already made. They hold where the cursor is **right now**, updated before every frame.

> You only **read** them. p5 writes them. A variable you made and one p5 made are the same idea — a name for a number — they just have different owners.

NAME	WHAT IS IN IT
`width · height`	set by `createCanvas`
`mouseX · mouseY`	where the cursor is, right now



a point p5 moves for you

Seeing a value — two windows LESSON 5

console.log() — for you

```
console.log(landerY);
console.log("got here");
```

Prints underneath the editor, in the **console** — not on the canvas. Nobody using your program ever sees it.

Reach for it when nothing moves, nothing happens, or it looks wrong. It answers *is the variable changing?* and *is this line running at all?*

text() — for the user

```
fill(120, 220, 140);
textSize(24);
text("ALTITUDE", 20, 40);
```

`text(what, x, y)` — what to show, then where. `fill()` sets its color, same as any shape.

> **The y is the bottom of the letters**, not the top. At `y = 0` the words are drawn above the canvas and you see nothing.

> **The console is for you. The canvas is for the user.** Both put a value where it can be read; only one is meant to be seen by anybody else.

Strings — what quotes do LESSON 5

```
text(landerY, 20, 40);   // 210
text("landerY", 20, 80); // landerY
```

Without quotes it is a **variable** and you get what is inside it. With quotes it is a **string** — just those characters.

Joining them

```
text("ALT " + landerY, 20, 40);
```

The `+` glues a string to a number. That is called **concatenation**.

> `"200"` and `200` are different things. `"200" + 1` gives `"2001"`, because for strings `+` means *join*, not *add*.

Events — code that runs when something happens LESSON 5

```
function keyPressed() {
  landerY = 40;
  speed = 2;
}
```

runs **once**, when a key goes down

```
function mousePressed() {
  speed = 0;
}
```

runs **once**, when the mouse is clicked

> **Three ways a handler silently does nothing.** No error message for any of them.
> - Written *inside* `draw()` instead of beside it

- Spelled `keypressed` — the capital P matters
- Misspelled any other way — p5 looks for that exact name

Why one line can reset a whole drawing. Every part of the rocket is written as an offset from `landerY`, so putting that variable back puts everything back. A drawing built from fixed numbers could not be reset at all.

Debugging — finding out what went wrong LESSON 6

An error stops the program and names the line. Loud, and easy.
A bug runs perfectly and does the wrong thing. Quiet, and hard.

Four questions, in order:

ASK	HOW
Is this line running at all?	`console.log("got here")`
Is this variable changing?	`console.log(landerY)`
Is it where I think it is?	inside `draw()`, or outside?
Is it spelled exactly right?	`keyPressed, not keypressed`

Symptoms you have met:

WHAT YOU SEE	USUALLY MEANS
Nothing moves at all	`let` inside `draw()`
A shape is missing	something drawn after it covers it
A shape vanished completely	it is above `background()`
Text is invisible	no `fill()` before `text()`
Everything down and right by half its size	`rectMode(CENTER)` missing
One part left behind	still a fixed number, not an offset
A handler does nothing	misspelled, or inside `draw()`

Change one thing, then run it. Two changes and you will not know which one worked — and you will not be able to do it again next time.

On the AP exam

What you write

```
let y = 100;
y = y + 1;
```

Exam Reference Sheet

```
y ← 100
y ← y + 1
```

The AP exam is not written in JavaScript. It uses College Board's own notation. The arrow `←` means **becomes** — the same word we use for `=`. No `let`, no semicolon. There is no canvas on the exam, so `DISPLAY(value)` takes one thing and shows it — no x, no y.

Unit map — what you did, in order

L1 Coordinates and lines
L2 Rectangles, triangles, color, order
L3 rectMode, ellipses, the rocket
L4A Variables, motion, the draw loop
L4B mouseX/mouseY, offsets, the drone
L5 Text, strings, events
L6 Debugging — errors vs bugs, the checklist
Assessment Build something that moves

Where this sits in AP CSP

Big Idea 3 — Algorithms and Programming · 30–35% of the multiple-choice exam, the largest single slice.

CODE	LEARNING OBJECTIVE
3.1	Variables and Assignments
3.3	Developing Algorithms

`3.4`	Strings
`1.4`	Identifying and Correcting Errors
`AAP-1.A`	Represent a value with a variable
`AAP-1.B`	Determine the value of a variable as a result of an assignment
`AAP-2.D`	Evaluate expressions that manipulate strings
`3.A`	*Skill:* generalize data sources through variables
`4.B`	*Skill:* determine the result of code segments

What is next. The program can move, report and respond — but it still cannot *decide* anything. It has no way to ask a question. Unit 2 adds conditionals: `if` the rocket has gone too far, stop it. Everything on this sheet is the vocabulary that makes those decisions mean something.

Keep this sheet. Every lesson from here on assumes you can find things on it without being told where they are.